AISCI5000

Foundation of Machine Learning and AI

Single-Layer Networks: Classification

Textbook:

- C. M. Bishop, H. Bishop, *Deep Learning: Foundations and Concepts*, Ch. 5 (2024).

Reference:

- James, Witten, Hastie, Tibshirani, Taylor, *An Introduction to Statistical Learning*, Ch. 4 (2023).
- Deisenroth, Faisal, Ong, *Mathematics for Machine Learning*, (2020).

Outline

- 5.1 Discriminant Functions
  - Two Classes & Multiple Classes
  - 1-of-K Coding
  - Least Squares Limitations
- 5.2 Decision Theory for Classification
  - Misclassification Rate & Expected Loss
  - Reject Option
  - Confusion Matrix, ROC, AUC
- 5.3 Generative Classifiers
  - Gaussian Class-Conditionals
  - Exponential Family
- 5.4 Discriminative Classifiers
  - Logistic & Multi-class Logistic Regression
  - Canonical Link Functions

Introduction

The goal in classification is to take an input vector $\mathbf{x} \in \mathbb{R}^D$ and assign it to one of K discrete classes $\mathcal{C}_k$ $(k = 1, \ldots, K)$.

- In the most common scenario, classes are disjoint.
- The input space is divided into *decision regions* bounded by *decision boundaries*.
- For **linear models**, decision surfaces are $(D-1)$-dimensional hyperplanes.
- Data sets whose classes can be separated exactly by linear surfaces are called *linearly separable*.

In this chapter, we will study three distinct classification approaches:

- Discriminant functions: assign each vector $\mathbf{x}$ directly to a specific class.
- Generative probabilistic models: model the class-conditional densities $p(\mathbf{x}|\mathcal{C}_k)$; along with the prior $p(\mathcal{C}_k)$, compute the posterior probabilities $p(\mathcal{C}_k|\mathbf{x})$ via Bayes' theorem.
- Discriminative probabilistic models: directly model $p(\mathcal{C}_k|\mathbf{x})$.

Among the three, generative probabilistic models tend to be the most demanding (require large data sets to train). Discriminant functions are the simplest but unlike the other two approaches, they do not yield posterior probabilities $p(\mathcal{C}_k|\mathbf{x})$. As we will learn later, knowing $p(\mathcal{C}_k|\mathbf{x})$ has many benefits.

5.1 Discriminant Functions

A discriminant is a function that takes an input variable $\mathbf{x}$ and assigns it to one of K classes.

5.1.1 Two classes

The simplest representation of a linear discriminant function is:

$$y(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0 \tag{1}$$

where $\mathbf{w}$ is the weight vector and w_0 is the bias parameter.

- Including a dummy input $x_0 = 1$ gives a compact notation:

$$\tilde{\mathbf{w}} = (w_0, \mathbf{w}^T)^T, \quad \tilde{\mathbf{x}} = (1, \mathbf{x}^T)^T \implies y(\mathbf{x}) = \tilde{\mathbf{w}}^T\tilde{\mathbf{x}} \tag{2}$$

- We assign $\mathbf{x}$ to $\mathcal{C}_1$ if $y(\mathbf{x}) \geq 0$, and to $\mathcal{C}_2$ otherwise.
- Decision boundary is defined by $y(\mathbf{x}) = 0$.

Geometry of Linear Discriminants

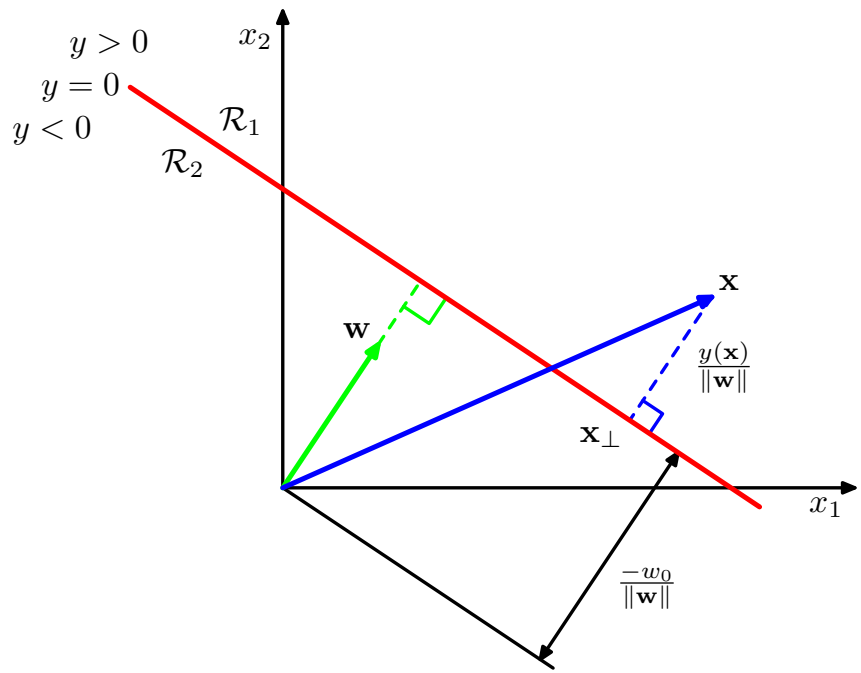
Recall that a subset $H \in \mathbb{R}^D$ is a hyperplane if and only if $H = [f : d]$ for some nonzero linear function f and some scalar d in $\mathbb{R}$. This means that setting a linear function to equal to a constant exactly removes one degree of freedom, resulting in a $D-1$ dimensional flat, *i.e.*, a hyperplane.

- A hyperplane divides $\mathbb{R}^D$ into two disjoint pieces.
- For points $\mathbf{x}_A, \mathbf{x}_B$ on the decision boundary: $\mathbf{w}^T(\mathbf{x}_A - \mathbf{x}_B) = 0 \implies \mathbf{w}$ is orthogonal to the boundary.
- $\mathbf{w}$ determines the orientation of the decision boundary.
- The signed distance from the decision boundary to the origin is:

$$\frac{\mathbf{w}^T\mathbf{x}}{\|\mathbf{w}\|} = -\frac{w_0}{\|\mathbf{w}\|} \tag{3}$$

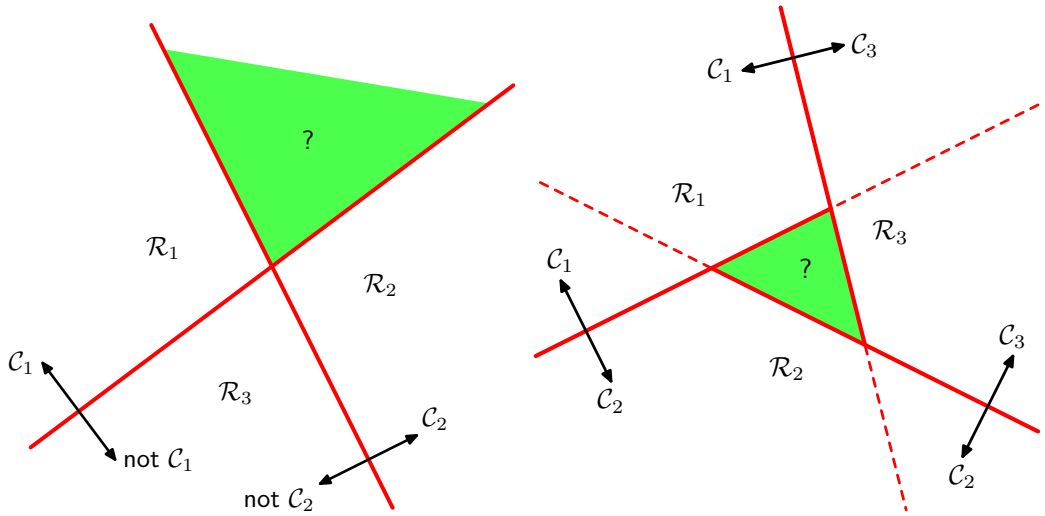
- w_0 determines the location of the decision boundary.
- The signed distance r from any point $\mathbf{x}$ to the decision boundary is:

$$\mathbf{x} = \mathbf{x}_\perp + r\frac{\mathbf{w}}{\|\mathbf{w}\|} \implies r = \frac{y(\mathbf{x})}{\|\mathbf{w}\|} \tag{4}$$



5.1.2 Multiple Classes ($K > 2$)

How do we divide the input space $\mathbb{R}^D$ into $K > 2$ regions (classes)?



What **NOT** to do (both approaches produce ambiguities):

- *One-versus-the-rest*: $K - 1$ classifiers, each solving a two-class problem of separating $\mathcal{C}_k$ from not-$\mathcal{C}_k$.
- *One-versus-one*: $K(K-1)/2$ classifiers, one for every possible pair of classes (separating $\mathcal{C}_k$ from $\mathcal{C}_j$).

What to do:

- Use a single K-class linear discriminant with K linear functions:

$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + w_{k0} \tag{5}$$

- Assign $\mathbf{x}$ to class $\mathcal{C}_k$ if $y_k(\mathbf{x}) > y_j(\mathbf{x})$ for all $j \neq k$.
- The decision boundary between $\mathcal{C}_k$ and $\mathcal{C}_j$ is therefore given by $y_k(\mathbf{x}) = y_j(\mathbf{x})$
- This decision boundary corresponds to a $(D-1)$-dimensional hyperplanes:

$$(\mathbf{w}_k - \mathbf{w}_j)^T \mathbf{x} + (w_{k0} - w_{j0}) = 0 \tag{6}$$

- Not all classes need to share a decision boundary, *i.e.*, a decision region $\mathcal{R}_k$ may not be a neighbour to $\mathcal{R}_j$ (as in a *Voronoi diagram*).

5.1.3 Target Coding: 1-of-K Scheme

Our discussions so far are about how, in principle, we may divide the input space $\mathbb{R}^D$ into K regions and through constructing discriminant functions ($y_k(\mathbf{x})$ or $y_k(\hat{\mathbf{x}})$), assign an input $\mathbf{x}$ to one of these regions. As in linear regression, we must use our observations (training set) to determine the parameters $\mathbf{w}_k$ in these constructed functions. But before that, we need to decide on how to represent target labels.

- Two classes: $t \in \{0, 1\}$ where $t = 1 \implies \mathcal{C}_1$ and $t = 0 \implies \mathcal{C}_2$.
- **1-of-K coding** (One-hot encoding): For $K > 2$, $\mathbf{t}$ is vector of length K.
- If $\mathbf{x} \in \mathcal{C}_k$, element $t_k = 1$ and all other $t_j = 0$. For example, given $K = 5$, an observation from class 2 will be represented as: $\mathbf{t} = (0, 1, 0, 0, 0)^T$.
- The target t_k is a discrete random variable $\in \{0, 1\}$. For a given observation, the value of t_k represents the empirical probability (either 0 or 1) of that observed data point belonging to class $\mathcal{C}_k$. The expectation value of t_k equals the probability $p(\mathcal{C}_k|\mathbf{x})$.

5.1.4 Least Squares for Classification

With these preparations, we are ready to study classification methods in details. But first, we may ask:

Can we apply the least squares (linear regression) method to classification?

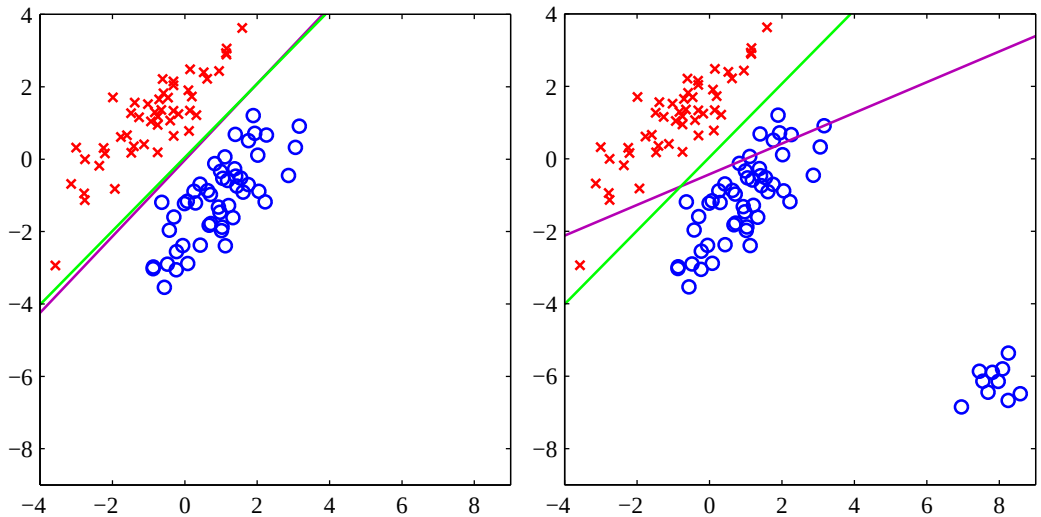
$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + w_{k0} \implies \mathbf{y}(\mathbf{x}) = \tilde{\mathbf{W}}^T \tilde{\mathbf{x}} \tag{7}$$

Similar to what we did in our previous lecture, by minimizing the sum-of-squares error function over training set $\{\mathbf{x}_n, \mathbf{t}_n\}$, we will obtain a closed-form solution for $\tilde{\mathbf{W}}$ (this time $\tilde{\mathbf{W}}$ is a matrix instead of a vector).

So yes, technically least squares can be applied. But it is typically NOT used to solve classification problems.

Limitations of Least Squares Classification

- **Elements of $\mathbf{y}$ are not probabilities**: If the linear constraint $\mathbf{a}^T\mathbf{t}_n + b = 0$ holds, our prediction satisfies $\mathbf{a}^T\mathbf{y}(\mathbf{x}) + b = 0$. Even though the elements of $\mathbf{y}$ sum to 1, they are not bounded in $[0, 1]$ and hence cannot be interpreted as probabilities.
- **Sensitivity to Outliers**: Least squares corresponds to maximum likelihood under Gaussian assumption. Binary targets are far from Gaussian, making least squares severely non-robust to outliers.



5.2 Decision Theory

Before we discuss more classification methods, let us revisit decision theory from previous lecture, this time in the context of classification.

Recall that predictions in machine learning can be divided into two stages:

1. **Inference**: Model the joint distribution $p(\mathbf{x}, \mathcal{C}_k)$ or conditional distribution $p(\mathcal{C}_k|\mathbf{x})$.
2. **Decision**: Make optimal class assignments based on inferred probabilities.

As an example, if we have taken an image of a skin lesion on a patient and wish to determine whether the patient has skin cancer, the input vector $\mathbf{x}$ is the set of pixel intensities in the image and the output variable t will represent the absence ($t = 0$) or presence ($t = 1$) of cancer, which we denote as class $\mathcal{C}_1$ and $\mathcal{C}_2$, respectively. Given Bayes' theorem:

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})} \tag{8}$$

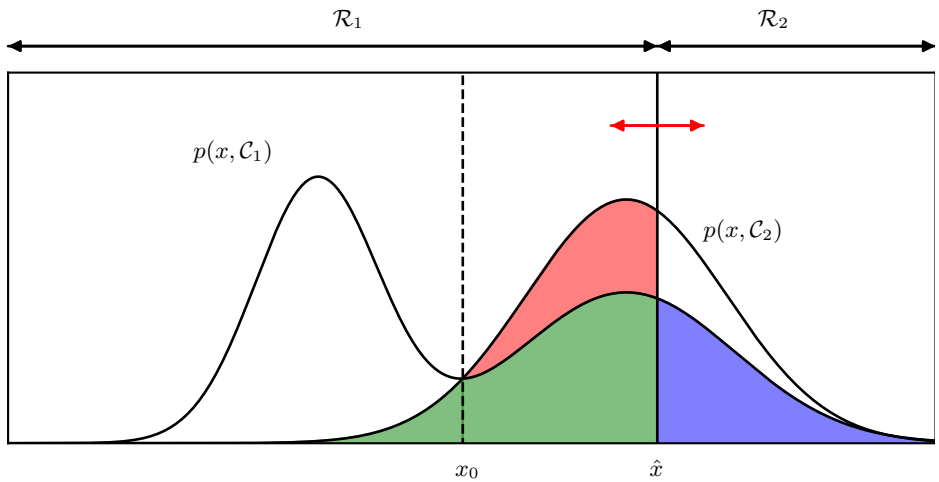
we can interpret $p(\mathcal{C}_k)$ as the prior probability of class $\mathcal{C}_k$, e.g., the fraction of population with skin cancer, and $p(\mathcal{C}_k|\mathbf{x})$ as the posterior probability after we are given a specific $\mathbf{x}$, i.e., after we have seen the patient's skin lesion image.

Estimating $p(\mathcal{C}_k|\mathbf{x})$ is at the heart of probabilistic models. This estimation is done at the inference stage. At the decision stage, we aim to make optimal decisions based on the inferred $p(\mathcal{C}_k|\mathbf{x})$.

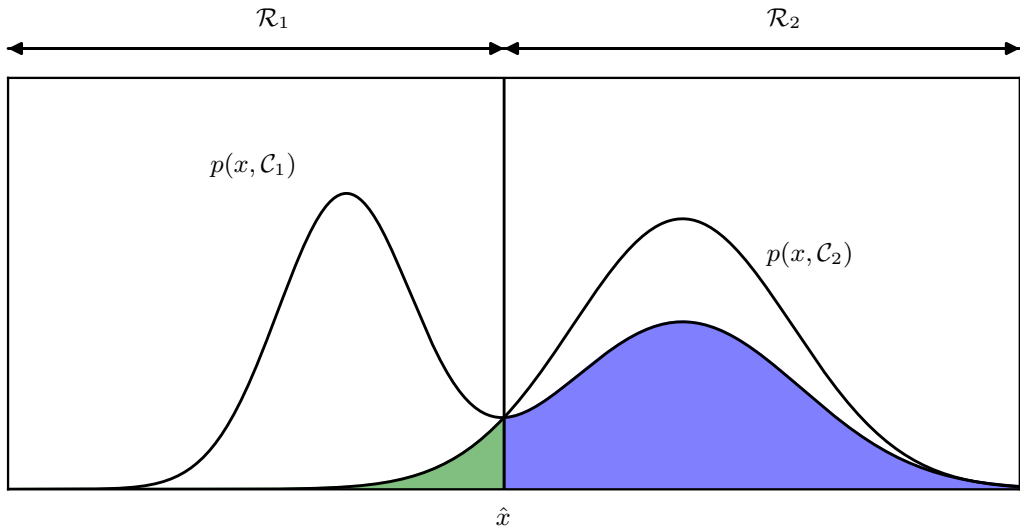
5.2.1 Misclassification Rate

Suppose that our goal is simply to make as few misclassifications as possible. Consider the cancer diagnosis example given earlier, the probability of mistake (misclassification) in this 2-class case is:

$$p(\text{mistake}) = \int_{\mathcal{R}_1} p(\mathbf{x}, \mathcal{C}_2) d\mathbf{x} + \int_{\mathcal{R}_2} p(\mathbf{x}, \mathcal{C}_1) d\mathbf{x} \tag{9}$$



That is, the mistakes are due to points from class $\mathcal{C}_2$ being misclassified as $\mathcal{C}_1$ (the first integral and the green+red areas) and points from class $\mathcal{C}_1$ being misclassified as $\mathcal{C}_2$ (the second integral and the blue area).



The sum of the green+blue areas is a constant—no matter where we draw the vertical line representing our decision boundary, this sum is unaffected (it reflects the intrinsic uncertainty of our classification problem). But we can eliminate the red area by placing the decision boundary where $p(\mathbf{x}, \mathcal{C}_1) = p(\mathbf{x}, \mathcal{C}_2)$. Thus, to minimize $p(\text{mistake})$, we assign $\mathbf{x}$ to $\mathcal{R}_1$ if $p(\mathbf{x}, \mathcal{C}_1) > p(\mathbf{x}, \mathcal{C}_2)$ and to $\mathcal{R}_2$ otherwise.

Since $p(\mathbf{x}, \mathcal{C}_k) = p(\mathcal{C}_k|\mathbf{x})p(\mathbf{x})$, we minimize the misclassification rate by assigning $\mathbf{x}$ to the class with the **largest posterior probability** $p(\mathcal{C}_k|\mathbf{x})$.

Loss Matrix

While minimizing the misclassification rate may appear to be a natural choice, it is in fact not always our goal in classification. This is because by simply minimizing the misclassification rate, we are treating all mistakes equally. In practice, the costs may differ depending on what types of mistakes we make (e.g., diagnosing cancer as healthy vs. healthy as cancer).

To formalize our considerations of different costs (loss) associated with different mistakes, we introduce a loss matrix: each element L_{kj} of this matrix represents the loss incurred by assigning an input $\mathbf{x}$ in the **true class** $\mathcal{C}_k$ to the *predicted class* $\mathcal{C}_j$.

	Normal $(j = 1)$	*Cancer* $(j = 2)$
Normal $(k = 1)$	0	1
Cancer $(k = 2)$	100	0

5.2.2 Expected Loss

The goal of classification becomes to minimize the **Expected Loss**:

$$\mathbb{E}[L] = \sum_k \sum_j \int_{\mathcal{R}_j} L_{kj} p(\mathbf{x}, \mathcal{C}_k) d\mathbf{x} \tag{10}$$

For a given input $\mathbf{x}$, we aim to assign it to an optimal decision region $\mathcal{R}_j$ such that after averaging over all input, $\mathbb{E}[L]$ achieves its minimum.

- A single input can only be assigned to one $\mathcal{R}_j$ (associated with column j in the loss matrix). Imagine going through the loss matrix column by column and ask, which column yields the smallest weighted sum $\sum_k L_{kj} p(\mathcal{C}_k|\mathbf{x})$?
- By assigning $\mathbf{x}$ to the above column, we minimize the loss associated with this single input. Since the input data are independent of each other, we can repeat this procedure and averaging over all $\mathbf{x}$ to achieve the minimized expected loss.

Therefore, given a loss matrix L_{kj}, the optimal decision rule is to assign each $\mathbf{x}$ to class j that minimizes $\sum_k L_{kj} p(\mathcal{C}_k|\mathbf{x})$.

5.2.3 The Reject Option

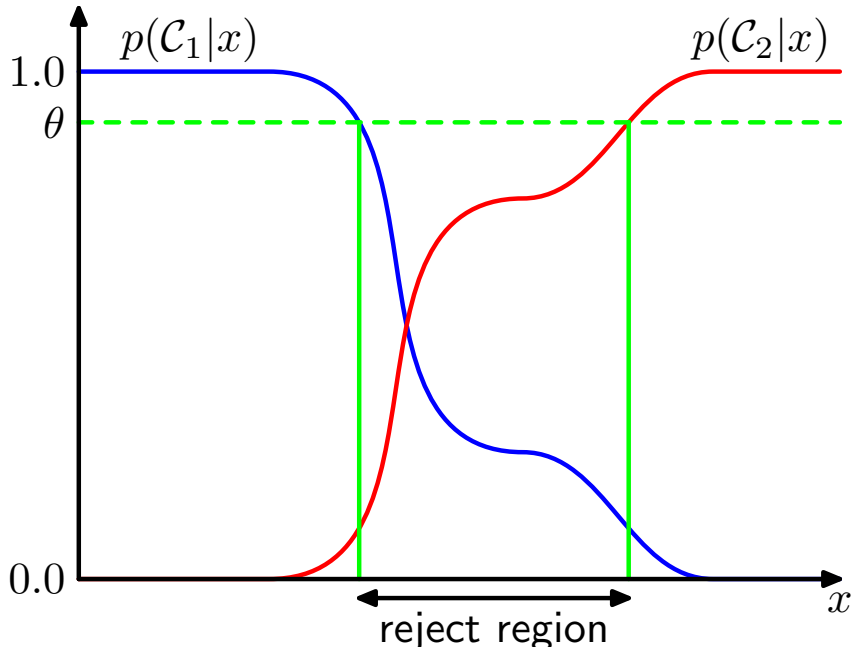
If even the largest $p(\mathcal{C}_k|\mathbf{x})$ is far from unity, or, there are regions where $p(\mathcal{C}_k|\mathbf{x})$ is comparable across classes, classification becomes significantly uncertain.

In some applications (e.g., the cancer diagnosis), it may be appropriate to avoid making decisions on these difficult cases—this is known as the *reject option*.

This can be achieved by introducing a threshold θ: we reject $\mathbf{x}$ if $\max_k p(\mathcal{C}_k|\mathbf{x}) \leq \theta$.

The fraction of rejected points is controlled by θ:

- Setting $\theta = 1$ rejects all inputs.
- Setting $\theta < 1/K$ rejects no inputs.



- With the reject option, we avoid automated decisions on ambiguous cases.
- Human intervention (e.g., requesting the patient to do a biopsy) may be needed to classify these cases.

5.2.4 Inference and Decision

As briefly mentioned at the beginning of this lecture, there are generally three distinct approaches to solving the classification problems:

- **Discriminant Functions**: Learn mapping $f(\mathbf{x})$ directly from inputs to class labels without explicit probabilities.
- **Generative Models**: Learn class-conditional densities $p(\mathbf{x}|\mathcal{C}_k)$ and priors $p(\mathcal{C}_k)$. Use Bayes' theorem to find $p(\mathcal{C}_k|\mathbf{x})$. This approach is *generative* because once we have learned $p(\mathbf{x}|\mathcal{C}_k)$, we can generate synthetic data by sampling from it.
- **Discriminative Models**: Model the conditional probabilities $p(\mathcal{C}_k|\mathbf{x})$ directly.

Before we study the latter two approaches, let us briefly discuss the benefit of computing the posterior probabilities and a few commonly used metrics to evaluate predictive performance of a classifier.

Why Compute Posterior Probabilities?

Even if decisions are our final goal, knowing $p(\mathcal{C}_k|\mathbf{x})$ is beneficial:

- **Minimizing risk**: Allows instant adjustment if loss matrix L_{kj} changes.
- **Reject option**: Allows thresholding based on our tolerance of uncertainty.
- **Compensating for class priors**: Allows adjusting our models trained on artificially balanced datasets:

$$p(\mathcal{C}_k|\mathbf{x}) \propto p_{\text{train}}(\mathcal{C}_k|\mathbf{x}) \frac{p_{\text{target}}(\mathcal{C}_k)}{p_{\text{train}}(\mathcal{C}_k)} \tag{11}$$

- **Combining models**: Allows probabilities from separate sources (e.g., imaging and blood test) to be merged systematically.

5.2.5 Classifier Accuracy

The simplest performance measure of a classifier is **accuracy**, defined as the fraction of correct predictions:

$$\text{Accuracy} = \frac{N_{TP} + N_{TN}}{N} = \frac{N_{TP} + N_{TN}}{N_{TP} + N_{FP} + N_{TN} + N_{FN}}, \tag{12}$$

where N_{TP}, N_{FP}, N_{TN} and N_{FN} stand for the number of true positive (TP), false positive (FP), true negative (TN) and false negative (FN), respectively, while $N = N_{TP} + N_{FP} + N_{TN} + N_{FN}$.

Note that accuracy can be misleading, especially on imbalanced datasets. For example, if 1 in 1,000 patients has cancer, a naive classifier predicting "normal" diagnosis for everyone gets 99.9% accuracy, even though this classifier is completely useless.

Confusion Matrix & Performance Metrics

We summarize predictions and true class labels in a **confusion matrix**:

	Predicted Normal	Predicted Cancer
True Normal	N_{TN}	N_{FP}
True Cancer	N_{FN}	N_{TP}

Common evaluation metrics derived from the matrix are:

$$\text{Precision} = \frac{N_{TP}}{N_{TP} + N_{FP}} \tag{13}$$

$$\text{Recall (Sensitivity, TPR)} = \frac{N_{TP}}{N_{TP} + N_{FN}} \tag{14}$$

$$\text{False Positive Rate (FPR)} = \frac{N_{FP}}{N_{FP} + N_{TN}} \tag{15}$$

$$\text{False Discovery Rate (FDR)} = \frac{N_{FP}}{N_{FP} + N_{TP}} \tag{16}$$

Although we demonstrate them using a binary classifier (healthy vs cancer), these metrics can be applied on multi-class ($K > 2$) classifiers.

Type I & Type II Errors

- **Type I Error** (N_{FP}): Predicting positive when the true class is negative (e.g., diagnosing a healthy person with cancer).

$$\text{Type I Error Rate } (\alpha) = \text{FPR} = \frac{N_{FP}}{N_{FP} + N_{TN}} \tag{17}$$

- **Type II Error** (N_{FN}): Predicting negative when the true class is positive (e.g., missing a cancer diagnosis).

$$\text{Type II Error Rate } (\beta) = \text{FNR} = \frac{N_{FN}}{N_{TP} + N_{FN}} \tag{18}$$

- Sensitivity $= 1 - \beta$:

$$\text{Sensitivity} = \text{TPR} = \frac{N_{TP}}{N_{TP} + N_{FN}} = 1 - \beta \tag{19}$$

- Specificity $= 1 - \alpha$:

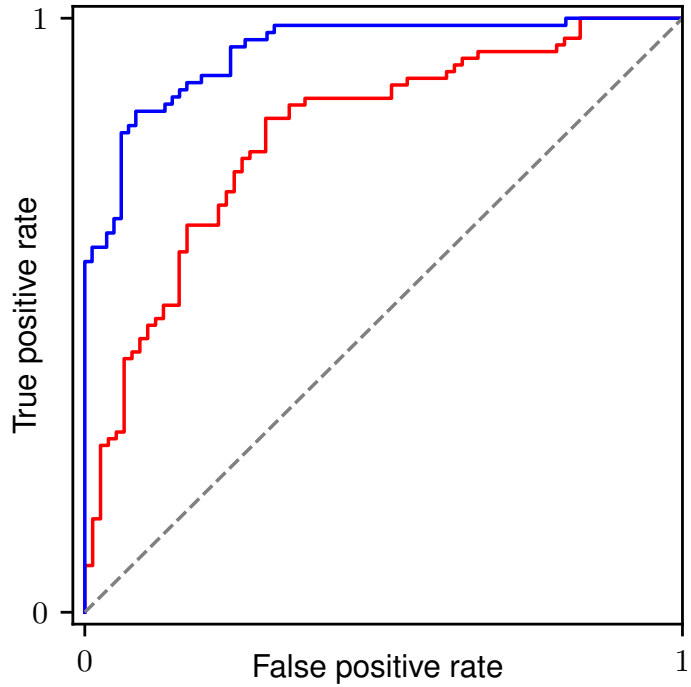
$$\text{Specificity} = \text{TNR} = \frac{N_{TN}}{N_{FP} + N_{TN}} = 1 - \alpha \tag{20}$$

5.2.6 Receiver Operating Characteristic (ROC)

A probabilistic classifier will output a posterior probability, which can be converted to a decision by setting a threshold.

As the value of the threshold is varied, we can examine the trade-off between α and β via the *receiver operating characteristic* (ROC) curve.

- The ROC curve plots TPR $(1 - \beta)$ vs. FPR (α) as the decision boundary moves—it represents **the same classifier evaluated at different decision thresholds**.
- Bottom-Left $(0, 0)$: predictions are all negative (no false positives, no true positives).
- Top-Right $(1, 1)$: predictions are all positive (no false negatives, no true negatives).
- Top-Left $(0, 1)$: predictions are perfect classification.



As a reference, we consider a random classifier that simply assigns each data point to cancer with probability ρ. As we vary ρ from 0 to 1, we trace out the diagonal dashed line that represents the *random baseline*. Any classifier below this line performs worse than random guessing.

Finally, if we wish to characterize a classifier's performance with a single scalar metric, we have:

- Area Under Curve (AUC): the total area under the ROC curve.
  - AUC $= 0.5 \implies$ Random guessing baseline.
  - AUC $= 1.0 \implies$ Perfect separation between classes.

- F-Score: the harmonic mean of precision and recall.

$$F = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2N_{TP}}{2N_{TP} + N_{FP} + N_{FN}} \tag{21}$$

The F-score (specifically, the F_1-score, an example of the more general F_β-score) is particularly useful for imbalanced problems where true negatives (N_{TN}) dominate (N_{TN} does not enter the calculation of F-score, preventing it from being artificially inflated as *accuracy*).

5.3 Generative Classifiers

We turn now to probabilistic models for classification. We will see how models with linear decision boundaries arise naturally from assumptions (shared covariance matrix) about the distribution of the data.

First, we consider problems with two classes. The posterior probability for class $\mathcal{C}_1$ is:

$$p(\mathcal{C}_1|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x})} = \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1) + p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} = \sigma(a) \tag{22}$$

where $\sigma(a)$ is the logistic sigmoid function:

$$\sigma(a) = \frac{1}{1 + \exp(-a)}, \quad a = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} \tag{23}$$

Note that the above is simply the Bayes' theorem applied on the two-class case.

The inverse of logistic sigmoid is the logit function (log odds): $a = \ln(\sigma/(1-\sigma))$.

Softmax Function for $K > 2$ Classes

For $K > 2$ classes, the multi-class posterior probability is:

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{\sum_j p(\mathbf{x}|\mathcal{C}_j)p(\mathcal{C}_j)} = \frac{\exp(a_k)}{\sum_j \exp(a_j)} \tag{24}$$

where $a_k = \ln(p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k))$.

The R.H.S. of the above equation is known as the *normalized exponential* or softmax function.

- It provides a smooth approximation to the *max* function.
- If $a_k \gg a_j$ for $j \neq k$, $p(\mathcal{C}_k|\mathbf{x}) \simeq 1$ and $p(\mathcal{C}_j|\mathbf{x}) \simeq 0$.

5.3.1 Gaussian Conditionals

The essence of all generative probabilistic models is to make assumptions about the class-conditional densities of the data. In the following, we will assume that the class-conditional densities are Gaussian with shared covariance matrix $\mathbf{\Sigma}$:

$$p(\mathbf{x}|\mathcal{C}_k) = \frac{1}{(2\pi)^{D/2}|\mathbf{\Sigma}|^{1/2}} \exp\left\{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^T\mathbf{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)\right\} \quad (25)$$

Recall that for a two-class problem ($K = 2$),

$$p(\mathcal{C}_1|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1) + p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} = \sigma(a) \quad (26)$$

where the pre-activation $a(\mathbf{x})$ is defined as the log-odds ratio:

$$a = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)} + \ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)} \quad (27)$$

Substituting the Gaussian densities $p(\mathbf{x}|\mathcal{C}_1)$ and $p(\mathbf{x}|\mathcal{C}_2)$ into the log-odds ratio, we have:

$$\begin{aligned}\ln \frac{p(\mathbf{x}|\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)} &= \ln p(\mathbf{x}|\mathcal{C}_1) - \ln p(\mathbf{x}|\mathcal{C}_2) \\ &= \left[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_1)^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_1)\right] - \left[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_2)^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_2)\right]\end{aligned}$$

Expanding the first pair of square brackets, we have

$$(\mathbf{x}-\boldsymbol{\mu}_1)^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_1) = \mathbf{x}^T\boldsymbol{\Sigma}^{-1}\mathbf{x} - 2\boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\mathbf{x} + \boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1$$

Note that we have used the symmetry property $\mathbf{x}^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 = \boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\mathbf{x}$. This is because every term in this expansion must be a scalar (Why?).

Expanding the second pair of square brackets and plugging them back into the log-odds ratio, we have:

$$\ln \frac{p(\mathbf{x}|\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)} = -\frac{1}{2}\left(\mathbf{x}^T\boldsymbol{\Sigma}^{-1}\mathbf{x} - 2\boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\mathbf{x} + \boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1\right) + \frac{1}{2}\left(\mathbf{x}^T\boldsymbol{\Sigma}^{-1}\mathbf{x} - 2\boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\mathbf{x} + \boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2\right)$$

Notice that the quadratic terms in $\mathbf{x}$ ($\mathbf{x}^T\boldsymbol{\Sigma}^{-1}\mathbf{x}$) cancel out perfectly because of the shared covariance matrix $\boldsymbol{\Sigma}$.

Grouping the remaining linear and constant terms, we arrive at:

$$\ln \frac{p(\mathbf{x}|\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)} = \underbrace{\left(\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)\right)^T\mathbf{x}}_{\text{Linear in } \mathbf{x}} - \frac{1}{2}\boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2$$

LDA

Finally, adding the log-prior ratio $\ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)}$ back into $a(\mathbf{x})$:

$$a(\mathbf{x}) = \left(\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)\right)^T \mathbf{x} - \frac{1}{2}\boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 + \ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)}$$

This is a linear function of $\mathbf{x}$, which can be compactly written as:

$$a(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0 \tag{28}$$

where:

$$\mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \tag{29}$$

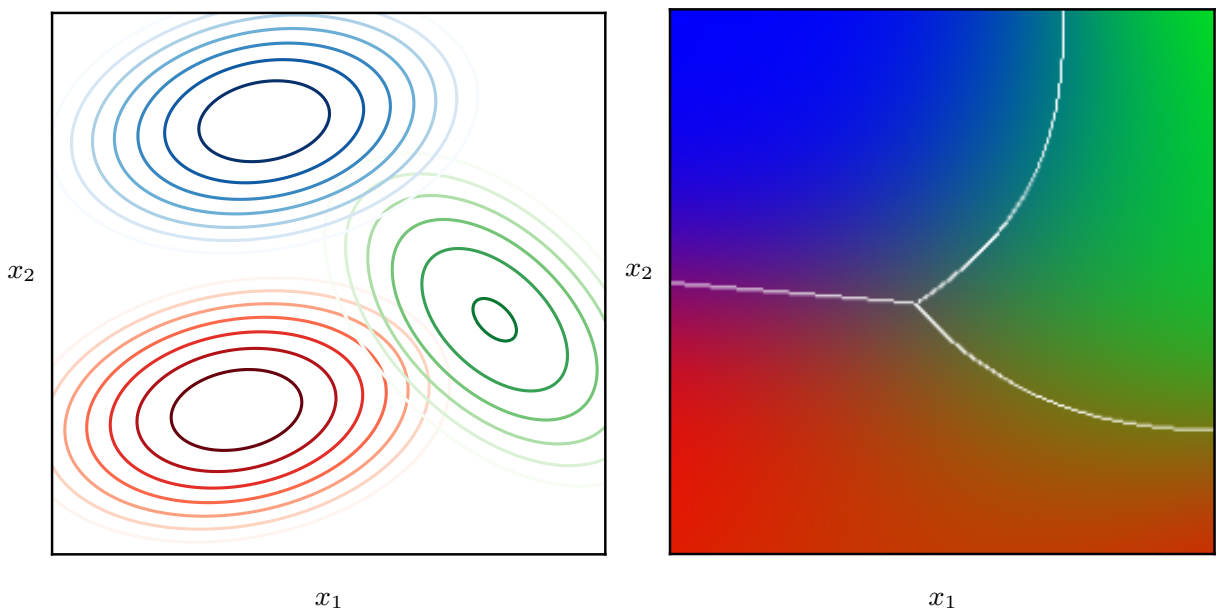
$$w_0 = -\frac{1}{2}\boldsymbol{\mu}_1^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 + \ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)} \tag{30}$$

The resulting decision boundaries are linear in input space, so this approach is also known as Linear Discriminant Analysis (LDA).

QDA

If we relax the assumption of a shared covariance matrix and allow each class-conditional density $p(\mathcal{C}_k|\mathbf{x})$ to have its own covariance matrix $\mathbf{\Sigma}_k$, then the earlier cancellation will no longer occur.

We will therefore obtain quadratic functions of $\mathbf{x}$, which give rise to *quadratic discriminant*, leading to the method known as Quadratic Discriminant Analysis (QDA).



5.3.2 Maximum Likelihood Solution

We are not done with LDA yet—we have established our Gaussian assumptions of the class-conditional densities (with shared $\boldsymbol{\Sigma}$), but we still need to determine their parameters. This is achieved via Maximum Likelihood.

Consider the two-class case and denoting the prior as $p(\mathcal{C}_1) = \pi$ (naturally $p(\mathcal{C}_2) = 1 - \pi$), the likelihood for our observations $\{\mathbf{x}_n, t_n\}$ is:

$$p(\mathbf{t}, \mathbf{X}|\pi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) = \prod_{n=1}^{N} [\pi \mathcal{N}_n(\boldsymbol{\mu}_1, \boldsymbol{\Sigma})]^{t_n} [(1-\pi)\mathcal{N}_n(\boldsymbol{\mu}_2, \boldsymbol{\Sigma})]^{1-t_n} \tag{31}$$

Maximizing the *log likelihood* with respect to π, $\boldsymbol{\mu}_1$ and $\boldsymbol{\mu}_2$ (respectively) yields:

$$\pi = \frac{N_1}{N_1 + N_2} \tag{32}$$

$$\boldsymbol{\mu}_1 = \frac{1}{N_1} \sum_{n=1}^{N} t_n \mathbf{x}_n, \quad \boldsymbol{\mu}_2 = \frac{1}{N_2} \sum_{n=1}^{N} (1 - t_n) \mathbf{x}_n \tag{33}$$

Maximizing with respect to the shared covariance matrix $\boldsymbol{\Sigma}$ yields:

$$\boldsymbol{\Sigma} = \mathbf{S} = \frac{N_1}{N}\mathbf{S}_1 + \frac{N_2}{N}\mathbf{S}_2 \tag{34}$$

So the shared covariance is a weighted average of individual sample covariances, where:

$$\mathbf{S}_1 = \frac{1}{N_1}\sum_{n\in\mathcal{C}_1}(\mathbf{x}_n - \boldsymbol{\mu}_1)(\mathbf{x}_n - \boldsymbol{\mu}_1)^T \tag{35}$$

$$\mathbf{S}_2 = \frac{1}{N_2}\sum_{n\in\mathcal{C}_2}(\mathbf{x}_n - \boldsymbol{\mu}_2)(\mathbf{x}_n - \boldsymbol{\mu}_2)^T \tag{36}$$

Plugging these results into Eqs.29-30, we have the full LDA model.

Just as linear regression, LDA has a closed-form solution.

5.3.4 Exponential Family

As it turns out, if we assume that the class-conditional densities are members of the **exponential family**:

$$p(\mathbf{x}|\boldsymbol{\lambda}_k, s) = \frac{1}{s} h\left(\frac{\mathbf{x}}{s}\right) g(\boldsymbol{\lambda}_k) \exp\left\{\frac{1}{s}\boldsymbol{\lambda}_k^T \mathbf{x}\right\} \tag{37}$$

and that the scaling parameter s is **shared across all classes**, then the posterior will always be given by a logistic sigmoid acting on a linear function. The resulting decision boundaries will be linear, so we will have a linear model for classification.

5.4 Discriminative Classifiers

Our third and final type of models in this chapter are discriminative models.

Instead of modeling $p(\mathbf{x}|\mathcal{C}_k)$ and combine them with the prior $p(\mathcal{C}_k)$ to yield the posterior $p(\mathcal{C}_k|\mathbf{x})$, we now model $p(\mathcal{C}_k|\mathbf{x})$ directly.

- Fewer parameters to estimate compared to generative models.
- Better predictive performance when class-conditional assumptions fail.

As a start, recall that the function $\sigma(a) = \frac{1}{1+\exp(-a)}$ introduced earlier is an example of an **activation function**. Its inverse is called a **link function** and in this case:

$$a = \ln\left(\frac{\sigma}{1-\sigma}\right) \tag{38}$$

For $K = 2$, we have already seen that $p(\mathcal{C}_1|\mathbf{x})$ can be cast as:

$$p(\mathcal{C}_1|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1) + p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} = \sigma(a) \tag{39}$$

$$a = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} \tag{40}$$

As noted earlier, a is simply the log odds:

$$a = \ln \frac{p(\mathcal{C}_1|\mathbf{x})}{p(\mathcal{C}_2|\mathbf{x})} = \ln \frac{p(\mathcal{C}_1|\mathbf{x})}{1 - p(\mathcal{C}_1|\mathbf{x})} \tag{41}$$

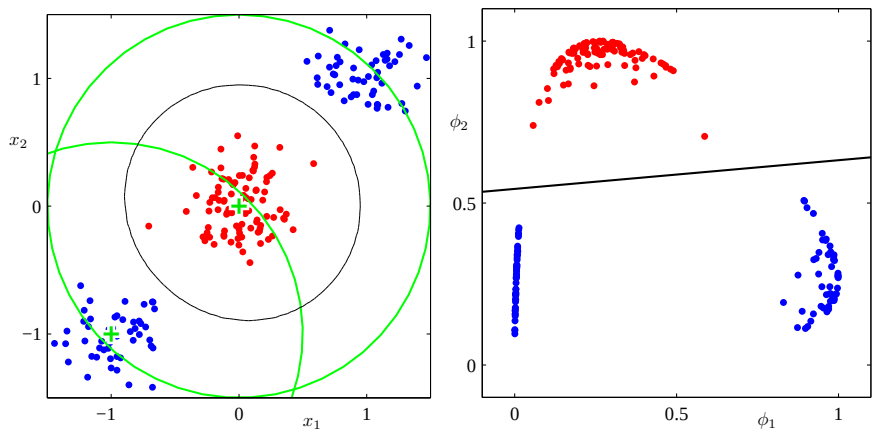
The essence of discriminative models is that instead of modeling the class-conditional densities and the priors given in Eq.40, we directly specify a functional form for the posterior distribution shown in Eq.41, and then link its parameters with a (generalized) linear model of the input. Specifically, for logistic regression, we assume that the target variable $t \in 0, 1$ follows the Bernoulli distribution and let the log odds be a linear combination of input ($a = \mathbf{w}^T\mathbf{x}$) or features as shown next.

5.4.2 Fixed Basis Functions

We can transform the input variables using fixed non-linear basis functions $\boldsymbol{\phi}(\mathbf{x})$, which helps separate non-linearly separable input:

$$y(\mathbf{x}, \mathbf{w}) = f(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})) \tag{42}$$

where $f(\cdot)$ is the **activation function**.
The resulting decision boundaries will be non-linear in input space $\mathbf{x}$. But as they are linear in the feature space $\boldsymbol{\phi}$, we still have a linear model.



Note that if there is any overlap in $p(\mathbf{x}|\mathcal{C}_k)$, then the posterior probabilities $p(\mathcal{C}_k|\mathbf{x})$ are not 0 or 1 for at least some $\mathbf{x}$, i.e., there is intrinsic uncertainty in our classification problem. This uncertainty cannot be eliminated by nonlinear transformations $\boldsymbol{\phi}(\mathbf{x})$.

5.4.3 Logistic Regression

The two-class model operating on feature vectors ϕ is known as the logistic regression model:

$$p(\mathcal{C}_1|\boldsymbol{\phi}) = y(\boldsymbol{\phi}) = \sigma(\mathbf{w}^T\boldsymbol{\phi}) \tag{43}$$

Note that the derivative of the logistic sigmoid is:

$$\frac{d\sigma}{da} = \sigma(1-\sigma) \tag{44}$$

The likelihood function for target dataset $\{\boldsymbol{\phi}_n, t_n\}$:

$$p(\mathbf{t}|\mathbf{w}) = \prod_{n=1}^{N} y_n^{t_n}\{1-y_n\}^{1-t_n}, \quad y_n = \sigma(\mathbf{w}^T\boldsymbol{\phi}_n) \tag{45}$$

Cross-Entropy Error Function

Taking the negative logarithm of likelihood gives the **Cross-Entropy Error**:

$$E(\mathbf{w}) = -\sum_{n=1}^{N}\{t_n \ln y_n + (1 - t_n)\ln(1 - y_n)\} \tag{46}$$

The gradient of the above error function with respect to $\mathbf{w}$ is:

$$\nabla E(\mathbf{w}) = \sum_{n=1}^{N}(y_n - t_n)\boldsymbol{\phi}_n \tag{47}$$

- Notice how the sigmoid derivative cancels perfectly in the above gradient.

- Unlike linear regression, there is no closed-form solution here due to nonlinearity in y_n.
- Nonetheless, the error function $E(\mathbf{w})$ is still **convex** (single global minimum).
- Its optimization can be achieved via Iterative Reweighted Least Squares (IRLS) or Stochastic Gradient Descent (SGD).

Over-fitting on Linearly Separable Data:

- The maximum likelihood solution drives $\|\mathbf{w}\| \to \infty$ to make sigmoid a sharp step function ($\sigma \to 0$ or 1).
- Severe over-fitting occurs regardless of data size N.
- This can be resolved by adding L_2 weight regularization penalty.

5.4.6 Canonical Link Functions

As it turns out, linear regression, logistic regression (and Poisson regression, etc) all belong to **Generalized Linear Models (GLMs)**.

These models can be built by assuming that the target variable distribution belongs to the exponential family:

$$p(t|\eta, s) = \frac{1}{s} h\left(\frac{t}{s}\right) g(\eta) \exp\left\{\frac{\eta t}{s}\right\} \tag{48}$$

The conditional expectation of the target variable is $y \equiv \mathbb{E}[t|\eta] = -s\frac{d}{d\eta} \ln g(\eta) \implies \eta = \psi(y)$.

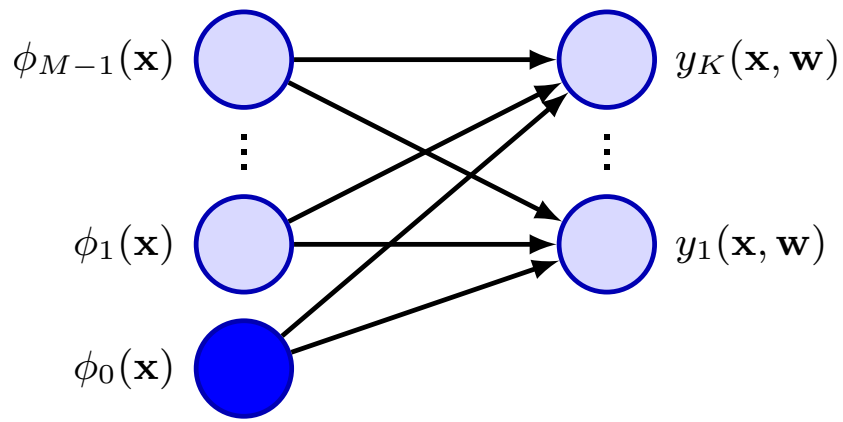
If we choose the activation function $f(a)$ such that $f^{-1}(y) = \psi(y)$ (**canonical link function**), then the gradient of the error function is guaranteed to simplify to:

$$\nabla E(\mathbf{w}) = \frac{1}{s} \sum_{n=1}^{N} (y_n - t_n)\boldsymbol{\phi}_n \tag{49}$$

Single-Layer Neural Network Representation

Classification can be viewed as a single-layer neural network:

- Input nodes represent basis functions $\phi_i(\mathbf{x})$.
- Output nodes represent predictions $y_k(\mathbf{x}, \mathbf{w})$.
- Links represent parameter weights w_{ki}.



As we will learn next, deep learning extends the above by abandoning fixed basis functions (learn them from data instead) and cascading multiple layers of adaptive weights.